Auction schedule on the announce: implementation plan

Worktree `auction-schedule`, branch `feat/auction-schedule`, off `announce-jo-branch` at b2290487. Written 2026-10-25 before any code change; approved 2026-10-25 with the four decisions of section 4.

1. Problem

Nothing on an announce says when its auction runs. The order module opens the auction on `AnnouncePublishedV1` with start = publishedAt and end = start + `auction.default-duration` (7 d) capped by expiresAt. Until that row exists (and it never does while the seller has no enabled payout account and `auction.seller-account.required=true`) `GET /api/auctions/{announceId}` is 404 and the panel shows "The auction is not open yet." with no date. A not-yet-started auction renders as "ended {endTime}" because the panel knows two time states only.

2. Survey facts (2026-10-25)

- `GoodsDetails` is an 8-field record built in five places: `AnnounceApiMapper.toDetails`, `AnnounceRows.toGoods`, `GoodsDetails.fixedPrice`, `AnnounceTest`, `AnnounceMessagingTest`.
- `AnnouncePublished` already carries `details`, but `Announce.apply(AnnouncePublished)` keeps the state's details; resolved dates written on the event would be lost in the aggregate unless apply switches to the event's details.
- The projection's publish handler updates the status columns of `announce` only; the resolved dates must also reach `announce_goods_detail`.
- `AnnounceGoodsDetailMapper.xml` holds a hand-written `upsert`; regeneration replaces the generated part, so the upsert is re-added afterwards (crawler-inventory gotcha: swap a restricted generator config, `mvn -P mbg mybatis-generator:generate` from db-reactive/, restore, `reactify_mapper.py`).
- `starts_epoch` and `ends_epoch` exist in `docker/cqrs/application.indexer-service.yml` with Result + Sort + Filter; `AnnounceDocument` fills them for events only.
- order: `auctionAnnounceEvents` computes start and end itself; `AuctionExtended` (event, apply, projection patch of end_time) is the model for a reschedule; `AuctionClosingJob` reads the row. `AnnounceEditedV1` is consumed only by the order snapshot subscriber; `AuctionOpenedV1` has no consumer outside the order module.
- SPA: `isAuctionOpen` already checks startTime; `AuctionPanel` renders two time states and a bare 404 state; `AnnounceCard.fromSummary` has no end date for goods; the admin `AuctionsPage` shows endTime only and has no detail screen.

3. Steps

1. announce-management domain and read model

- `GoodsDetails` gains `auctionStartsAt`, `auctionEndsAt` (LocalDateTime, nullable, last two fields) and a `withSchedule(start, end)` copy method. The record stays a plain value; the rules live in `AnnounceRules`.
- `AnnounceRules.checkDetails`: a schedule on a non-AUCTION goods, a job or an event answers 409 `auction-schedule-not-allowed`; start >= end answers 409 `auction-schedule-order`. `checkPublish`: end after expiresAt answers 409 `auction-ends-after-expiry`, a resolved end in the past 409 `auction-already-over`. New `resolveSchedule(goods, publishedAt, expiresAt, defaultDuration)`: null start = publishedAt, null end = start + duration capped by expiresAt.
- `AnnounceHandlers.publish` and `relist` resolve the schedule into `AnnouncePublished.details`; `Announce.apply(AnnouncePublished)` takes the event's details. Property `announce.auction.default-duration` (7 d) injected next to the Clock.

- Schema: `announce_goods_detail.auction_starts_at`, `auction_ends_at` (datetime null) in announce_table.sql, build script rerun, `AnnounceGoodsDetail` regenerated + upsert re-added; `selectSummaries` gains gd.auction_ends_at; `AnnounceSummaryRow` and `AnnounceSummary.KindFacts` gain auctionEndsAt.
- Projection: the AnnouncePublished handler writes the status and then upserts the goods detail row; `AnnounceRows.toGoodsRow` / `toGoods` map the columns.
- Tests (AnnounceTest, AnnounceControllerTest, read-model test): explicit schedule published as given; none -> publish time + 7 d capped by expiry; end before start, schedule on FIXED_PRICE, end in the past rejected with the codes above.

2. API module and search document

- `GoodsDto` gains auctionStartsAt, auctionEndsAt; `AnnounceSummaryResponse` gains auctionEndsAt (appended); `AnnouncePublishedV1` and `AnnounceEditedV1` gain both (nullable, additive). `AnnounceMessaging.toPublished/toEdited` and `AnnounceApiMapper` map them.
- `AnnounceDocument.of` emits starts_epoch and ends_epoch for AUCTION goods. No indexer yml change (decision 4).

3. order module

- `auctionAnnounceEvents` uses the event's dates when both are present, today's rule otherwise.
- New command `RescheduleAuction(announceId, startsAt, endsAt)`; rule: OPEN, no standing bid, start < end (409 `auction-not-open` / `bids-exist` / `auction-window`); event `AuctionRescheduled(previousStartsAt, previousEndsAt, startsAt, endsAt, at)`; apply; projection patches start_time and end_time and logs an audit row; `AuctionService.reschedule`.
- Subscriber on `AnnounceEditedV1`: when the event carries dates, reschedule; no-op when no auction exists, warn on rejection.
- `AuctionRescheduledV1` in order-api, relayed to auction.v1.
- Tests: AuctionTest (reschedule before a bid, refused after a bid and when closed); OrderModuleEndToEndTest publishes with explicit dates and asserts the row; the fallback stays covered by the existing flow.

4. Customer SPA (gateway-api)

- `KindForms` goods: two datetime-local pickers shown when AUCTION is chosen; `validation.ts` mirrors the rules; emptyRequest / toRequest carry the fields.
- `AnnounceDetail`: "Auction: starts {date}, ends {date}" for AUCTION goods.
- `AuctionPanel`: before start ("starts {date}", bid form disabled), open ("started {date} · ends in {time}"), closed ("ended {date}"); the 404 state shows the announce's schedule plus "bidding opens once the seller's payout account is enabled".
- `AnnounceCard.fromSummary`: "Auction · ends {date}" from auctionEndsAt for local auctions.
- i18n en, da, sv; specs for AuctionPanel (three time states + 404) and validation (end before start).

5. Admin SPA (gateway-admin)

- `AuctionsPage` gains a start column next to the end column. No detail screen exists.

6. Docs and report

- doc/order-context.md §4 (opening and rescheduling rule), doc/announce-management-context.md §8 (fields, rules, columns), doc/frontend-edges.md §7 checklist entry, session report PDF in doc/.

4. Decisions (approved 2026-10-25)

#	Decision	Why
1	Relist re-resolves the schedule when the stored end has passed.	A relisted auction announce opens a fresh 7-day auction instead of one already over.

#	Decision	Why
2	Edits after publish keep explicit dates but revalidate them against expiresAt and the clock.	An edit cannot move the end into the past.
3	Schedule fields on a draft are stored as posted and resolved at publish only.	The SPA shows "starts {date}" only when the seller set one.
4	No indexer yml change.	starts_epoch and ends_epoch already have Result + Sort + Filter.

5. Verification

- `mvn -pl db-reactive test`
- `mvn -pl announce-management-api,announce-management -P '!webapp' test`
- `mvn -pl order -P '!webapp' test`
- `npm test` and `npm run i18n:check` in gateway-api and gateway-admin
- Manual on the stack: post an auction starting tomorrow, publish, open the detail and see the start date; as another member see the disabled bid form with "starts ..."; recorded in doc/frontend-edges.md §7.

6. Out of scope

Reserve and buy-now prices on the announce, auction templates, time zones beyond the server's local time, changing auction.default-duration semantics, notifications about the auction.

Status: approved 2026-10-25, implementation started with step 1.